

Atyani ERP — Frontend Developer Guide

> Complete reference for building the Tally-clone ERP frontend. Every sidebar module, sub-module, form field, and API call is documented here. No questions needed.

****Base URL:**** `https://apidevelop.yanibooks.com`

****Auth:**** All requests (except login/register) require header: `Authorization: Bearer <token>`

****Token storage:**** Save `accessToken` and `refreshToken` from login response in localStorage.

Global Conventions

API Response Format

```json

// Success

```
{ "success": true, "message": "...", "data": { ... }, "timestamp": "..." }
```

// List (paginated)

```
{ "success": true, "data": [...], "pagination": { "total": 150, "page": 1, "limit": 20, "total_pages": 8 } }
```

// Error

```
{ "success": false, "message": "...", "error_code": "VALIDATION_ERROR", "errors": [...] }
```

```

Date Format

Always use `YYYY-MM-DD` (e.g., `2026-03-31`).

Pagination

Query params: `?page=1&limit=20`

Auto Token Refresh (HTTP Interceptor — implement once globally)

1. If API returns 401 → call `POST /api/auth/refresh` with `{ "refresh\_token": "<stored>" }`
2. Success → update `accessToken` in storage → retry original request
3. Refresh fails → redirect to `/login`

Common Status Badge Colors

Status	Color
--------	-------

```
|---|---|
| DRAFT / PENDING | yellow |
| POSTED / ACTIVE / CONFIRMED / COMPLETED | green |
| CANCELLED / REJECTED / OBSOLETE / TERMINATED | red |
| PAID / CLEARED | blue |
| PARTIALLY_PAID / IN_PROGRESS / HALF_DAY | orange |
```

Sidebar Structure

...

```
|— 🏠 Dashboard
|— 🏢 Company & System Setup
|   |— Company Profile
|   |— Financial Years
|   |— Users & Access
|   |— Roles & Permissions
|   |— Data Backup
|— 🟡 Accounting & Finance
|   |— Chart of Accounts
|   |   |— Ledger Groups
|   |   |— Ledgers
|   |— Voucher Entry
|   |— Bank Reconciliation
|   |— Interest Calculation
|   |— Cost Centres
|   |— Budgets
|— 📄 Invoicing & Billing
|   |— Sales Invoices
|   |— Purchase Invoices
|   |— Credit Notes (Sales Returns)
|   |— Debit Notes (Purchase Returns)
|   |— Proforma Invoices
|   |— Sales Orders
|   |— Purchase Orders
|— 📦 Inventory Management
|   |— Stock Items
|   |— Stock Summary
|   |— Stock Transfer
|   |— Wastage
|   |— Stock Verification
```

- └─ HSN / SAC Codes
- └─  Manufacturing
 - └─ Bill of Materials (BOM)
 - └─ Production Orders
 - └─ Job Work
- └─  GST & Taxation
 - └─ GST Reports
 - └─ Reverse Charge (RCM)
- └─  Payroll & HR
 - └─ Employees
 - └─ Attendance
 - └─ Salary Structure
 - └─ Payslips
- └─  Banking & Payments
 - └─ Bank Reconciliation
 - └─ Post-Dated Cheques
- └─  Reports & MIS
 - └─ Trial Balance
 - └─ Profit & Loss
 - └─ Balance Sheet
 - └─ Ledger Account
 - └─ Outstanding Receivables
 - └─ Outstanding Payables
 - └─ Stock Summary
 - └─ Sales Analysis
 - └─ Purchase Analysis
 - └─ GST Summary
 - └─ Cash Flow
 - └─ Fund Flow
 - └─ Cash/Bank Book
 - └─ Day Book
- └─  Multi-Currency
- └─  Approval Workflow
- └─  Settings
 - └─ Company Settings
 - └─ Financial Years
 - └─ Voucher Types
 - └─ Warehouses

...

1. DASHBOARD

****Route:**** `/dashboard`

Page Load – Call these 3 APIs in parallel:

```

GET /api/dashboard/summary

GET /api/dashboard/sales-chart?months=6

GET /api/dashboard/top-customers?limit=5&from\_date=<fy\_start>&to\_date=<fy\_end>

```

KPI Cards (from `summary` response)

| Card | Field | Click Action |

|---|---|---|

| Total Receivables | `financials.total\_receivables` | → Outstanding Receivables report |

| Total Payables | `financials.total\_payables` | → Outstanding Payables report |

| Net Position | `financials.net\_position` | – |

| Month Sales | `financials.month\_sales` | → Sales Analysis |

| Month Purchases | `financials.month\_purchases` | → Purchase Analysis |

| Low Stock Alerts | `alerts.low\_stock\_items` | → Stock Verification |

| Pending Cheques | `alerts.pending\_cheques` | → Post-Dated Cheques |

Sales vs Purchase Chart

Bar/line chart from `sales-chart` response array: `[ { month: "2026-01", sales: 250000, purchases: 180000 } ]`

Top Customers Table

From `top-customers` response: Customer Name, Total Sales, Invoice Count.

Recent Invoices Table

From `summary.recent\_invoices`: Invoice No, Date, Customer, Amount, Payment Status.

2. COMPANY & SYSTEM SETUP

2.1 Company Profile

****Route:**** `/company/profile`

Page Load

...

GET /api/company

...

Show company details in read mode. "Edit" button toggles form.

Edit Company Form

Field	Input	Required	Key	Validation
Company Name	text	Yes	`name`	min 2 chars
Legal Name	text	No	`legal_name`	
GSTIN	text	No	`gstIn`	`^[0-9]{2}[A-Z]{5}[0-9]{4}[A-Z][1-9A-Z]Z[0-9A-Z]\$\`
PAN	text	No	`pan`	`^[A-Z]{5}[0-9]{4}[A-Z]\$\`
Address	textarea	No	`address`	
City	text	No	`city`	
State	text	No	`state`	
State Code	text	No	`state_code`	2 digits (e.g. 27)
Pincode	text	No	`pincode`	6 digits
Phone	text	No	`phone`	
Email	email	No	`email`	

Save:

...

PUT /api/company/{company_id}

Body: { name, legal_name, gstin, pan, address, city, state, state_code, pincode, phone, email }

...

Multi-Company Switch

...

GET /api/company/my-companies → dropdown of accessible companies

POST /api/auth/switch-company Body: { "company_id": <id> }

...

On switch success: replace stored `accessToken` with new one from response.

Create Additional Company (from dropdown → "+ Add Company")

...

POST /api/company

Body: { name, legal_name, gstin, pan, address, city, state, state_code, pincode, phone, email }

```

This seeds: 24 ledger groups, 12 ledgers, 9 voucher types, 1 financial year, 1
warehouse automatically.

---
```

2.2 Financial Years

```
**Route:** `/company/financial-years`
```

List

```


```

```
GET /api/settings/financial-years
```

```


```

```
Table: Start Date, End Date, Active (badge).
```

Create Form

```
| Field | Input | Required | Key |
```

```
| ---|---|---|---|
```

```
| Start Date | date | Yes | `start_date` |
```

```
| End Date | date | Yes | `end_date` |
```

```
| Set as Active | toggle | No | `is_active` |
```

```


```

```
POST /api/settings/financial-years
```

```
Body: { "start_date": "2026-04-01", "end_date": "2027-03-31", "is_active": true }
```

```


```

```
---
```

2.3 Users & Access

```
**Route:** `/company/users`
```

List

```


```

```
GET /api/users
```

```


```

```
Table: Name, Email, Role(s), Status, Actions (Edit, Toggle Status, Reset Password).
```

Create User Form

Field	Input	Required	Key	Notes
Full Name	text	Yes	`name`	
Email	email	Yes	`email`	must be unique
Password	password	Yes	`password`	min 8 chars
Phone	text	No	`phone`	
Assign Role(s)	multi-select	Yes	`role_ids[]`	load from `GET /api/roles`

```

...

POST /api/users
Body: { "name": "Alice", "email": "alice@co.com", "password": "Pass@123", "role_ids":
[2], "phone": "9876543210" }
...

#### Edit User
...

GET /api/users/{id}           → pre-fill form
PUT /api/users/{id}           Body: { name, phone }
PUT /api/users/{id}/roles     Body: { "role_ids": [1, 2] }
...

#### Toggle Status
...

POST /api/users/{id}/toggle-status
...

#### Reset Password (admin)
...

POST /api/users/{id}/reset-password
Body: { "new_password": "NewPass@123" }
...

---

### 2.4 Roles & Permissions

**Route:** `/company/roles`

#### List
...

GET /api/roles

```

```

    ...

Table: Role Name, Actions (Edit Permissions, Delete).

#### Permission Matrix (click a role to edit)
    ...

GET /api/roles/{id}
    ...

Render checkbox grid:

| Module | Create | Read | Update | Delete |
|---|---|---|---|---|
| accounting | ☐ | ☒ | ☐ | ☐ |
| sales_invoice | ☒ | ☒ | ☒ | ☐ |
| purchase_invoice | ... | | | |

**Available modules:** `accounting`, `sales_invoice`, `purchase_invoice`, `customers`,
`vendors`, `inventory`, `payroll`, `reports`, `settings`, `banking`, `manufacturing`,
`approvals`

**Save:**
    ...

PUT /api/roles/{id}/permissions
Body: {
  "permissions": [
    { "module": "sales_invoice", "can_create": true, "can_read": true, "can_update":
true, "can_delete": false }
  ]
}
    ...

#### Create Role
    ...

POST /api/roles
Body: { "name": "Accountant", "description": "Manages vouchers and ledgers" }
    ...

#### Delete Role
    ...

DELETE /api/roles/{id}
    ...

---
```


2.5 Data Backup

****Route:**** `/company/backup`

List Backups

```

GET /api/backups

```

Table: Filename, Size, Created At, Actions (Download, Delete).

Create Backup

```

POST /api/backups

```

Show spinner. Refresh list on success.

Download

```

GET /api/backups/{filename}/download

```

Trigger browser file download.

Delete

```

DELETE /api/backups/{filename}

```

3. ACCOUNTING & FINANCE

3.1 Chart of Accounts - Ledger Groups

****Route:**** `/accounting/ledger-groups`

List (Tree View recommended)

```

GET /api/ledger-groups

```

Display as expandable tree. Each node: Name, Nature badge.

****Default groups (seeded, non-deletable):****

Capital Account, Loans (Liability), Current Liabilities, Fixed Assets, Current Assets, Investments, Income, Expenses (Direct), Expenses (Indirect), Reserves & Surplus, Sundry Creditors, Duties & Taxes, Provisions, Sundry Debtors, Cash-in-Hand, Bank Accounts, Loans & Advances (Asset), Deposits (Asset), Stock-in-Hand, Sales Accounts, Other Income, Purchase Accounts, Direct Expenses, Indirect Expenses.

Create Ledger Group Form

Field	Input	Required	Key
Name	text	Yes	`name`
Nature	select	Yes	`nature` – ASSET / LIABILITY / INCOME / EXPENSE
Under (Parent)	select	No	`parent_group_id` – dropdown from existing groups

...

POST /api/ledger-groups

Body: { "name": "Travel Expenses", "nature": "EXPENSE", "parent_group_id": 9 }

...

Edit / Delete

...

PUT /api/ledger-groups/{id}

DELETE /api/ledger-groups/{id}

...

3.2 Chart of Accounts – Ledgers

****Route:**** `/accounting/ledgers`

List

...

GET /api/ledgers?group_id=&search=

...

Table: Name, Group, Nature, Opening Balance.

Create Ledger Form

Field	Input	Required	Key	Notes
Ledger Name	text	Yes	`name`	
Under Group	searchable select	Yes	`ledger_group_id`	from `GET /api/ledger-groups`
Opening Balance	number	No	`opening_balance`	default 0
Balance Type	radio	No	`balance_type`	DR or CR
GSTIN	text	No	`gstIn`	for party ledgers
Phone/Mobile	text	No	`mobile`	
Address	textarea	No	`address`	

```

...
POST /api/ledgers
Body: { "name": "HDFC Bank", "ledger_group_id": 16, "opening_balance": 100000,
"balance_type": "DR" }
...

#### Edit / Delete
...

PUT /api/ledgers/{id}
DELETE /api/ledgers/{id}
...

---

### 3.3 Voucher Entry

**Route:** `/accounting/vouchers`

#### List
...

GET
/api/vouchers?page=1&limit=20&from_date=2025-04-01&to_date=2026-03-31&status=POSTED&voucher_type_id=1
...

On page load also: `GET /api/settings/voucher-types` → populate Type filter.

Table: Voucher No, Date, Type, Narration, Dr Amount, Cr Amount, Status, Actions.

#### Create Voucher Form

**Step 1 – Header**

```

Field	Input	Required	Key
Voucher Type	select	Yes	`voucher_type_id` - from `GET /api/settings/voucher-types`
Date	date	Yes	`voucher_date`
Reference No	text	No	`reference_number`
Narration	textarea	No	`narration`

Voucher Types Reference:

Name	Use When
Payment	Paying someone (Cash/Bank going out)
Receipt	Receiving money (Cash/Bank coming in)
Journal	Non-cash adjustments, corrections
Contra	Transfer between cash/bank accounts
Sales	Invoice-backed sales entry
Purchase	Invoice-backed purchase entry
Credit Note	Sales return
Debit Note	Purchase return
Stock Journal	Stock adjustments

Step 2 - Entries (Double-Entry Lines)

Each row:

Field	Input	Required	Key	Notes
Ledger	searchable select	Yes	`ledger_id`	search from `GET /api/ledgers`
Dr/Cr	radio	Yes	`dr_cr`	DR or CR
Amount	number	Yes	`amount`	
Cost Centre	select	No	`cost_centre_id`	from `GET /api/cost-centres`

****Balance Check:**** Show live `Total DR = ₹X` vs `Total CR = ₹Y`. Disable "Save" button if DR ≠ CR.

Minimum: 2 rows required.

Save as Draft:

...

POST /api/vouchers

Body: {

```

  "voucher_type_id": 1,
  "voucher_date": "2026-03-15",

```

```

    "narration": "Office rent payment",
    "entries": [
      { "ledger_id": 5, "dr_cr": "DR", "amount": 25000 },
      { "ledger_id": 10, "dr_cr": "CR", "amount": 25000 }
    ]
  }
}
...

**Post Voucher (finalize – creates accounting effect):**
...

POST /api/vouchers/{id}/post
...

**Cancel Voucher:**
...

POST /api/vouchers/{id}/cancel
...

---

### 3.4 Bank Reconciliation

**Route:** `/accounting/bank-reconciliation`

#### List
...

GET /api/bank-reconciliations?bank_account_id=1&status=PENDING&limit=10
...

> First load: `GET /api/ledgers` filtered to Bank Accounts group → populate bank
selector.

Table: Bank Account, Statement Date, Statement Balance, Status, Actions.

#### Create Reconciliation Form

| Field | Input | Required | Key |
|---|---|---|---|
| Bank Account | select | Yes | `bank_account_id` – ledgers under "Bank Accounts"
group |
| Statement Date | date | Yes | `statement_date` |
| Closing Balance (as per statement) | number | Yes | `statement_balance` |

```

```

**Add Bank Statement Transactions (rows):**

| Field | Input | Key |
|---|---|---|
| Transaction Date | date | `transaction_date` |
| Cheque/Ref No | text | `cheque_no` |
| Description | text | `description` |
| Debit (money in bank) | number | `debit_amount` |
| Credit (money out of bank) | number | `credit_amount` |

...

POST /api/bank-reconciliations
Body: {
  "bank_account_id": 1,
  "statement_date": "2026-03-31",
  "statement_balance": 150000,
  "transactions": [
    { "transaction_date": "2026-03-30", "description": "Customer payment",
      "debit_amount": 50000, "credit_amount": 0 }
  ]
}
...

#### Matching Screen (open a reconciliation → `/bank-reconciliation/{id}`)
...

GET /api/bank-reconciliations/{id}/unmatched → bank entries not yet matched to ERP
vouchers
GET /api/bank-reconciliations/{id}/matched → already matched pairs
GET /api/bank-reconciliations/{id} → reconciliation summary
...

**Two-panel view:** Bank entries (left) | ERP Vouchers (right). User selects pairs and
clicks "Match".

**Match:**
...

POST /api/bank-reconciliations/match-transaction
Body: { "reconciliation_id": 1, "bank_transaction_id": 5, "voucher_id": 100 }
...

**Unmatch:**
...

```

```

POST /api/bank-reconciliations/unmatch-transaction
Body: { "reconciliation_id": 1, "bank_transaction_id": 5 }
...

**Finalize (lock reconciliation):**
...

POST /api/bank-reconciliations/{id}/finalize
...

**Summary for an account:**
...

GET /api/bank-reconciliations/account/{bank_account_id}/summary
...

---

### 3.5 Interest Calculation

**Route:** `/accounting/interest`

Two tabs: **On Receivables** | **On Payables**

#### Receivables Filter

| Field | Input | Key | Default |
|---|---|---|---|
| Annual Rate % | number | `rate` | 18 |
| As of Date | date | `as_of_date` | today |
| From Date | date | `from_date` | optional |
| To Date | date | `to_date` | optional |
| Customer | searchable select | `customer_id` | optional |

...

GET /api/interest/receivables?rate=18&as_of_date=2026-03-31&customer_id=1
...

**Response display:**
- Summary card: Total Outstanding, Total Interest, Total Due
- Detail table: Invoice No, Customer, Invoice Date, Days Outstanding, Outstanding Amt,
Interest Amt, Total Due

#### Payables Filter (same, but for vendors)

```

```

GET /api/interest/payables?rate=18&as_of_date=2026-03-31&vendor_id=1

```

```
---
```

3.6 Cost Centres

****Route:**** `/accounting/cost-centres``

Two sections (tabs): ****Categories**** | ****Cost Centres****

List Categories

```

GET /api/cost-centres/categories

```

Create Category Form

Field	Input	Required	Key
Name	text	Yes	<code>`name`</code>
Description	textarea	No	<code>`description`</code>

```

POST /api/cost-centres/categories
Body: { "name": "Operations", "description": "..."}

```

Edit/Delete Category

```

PUT /api/cost-centres/categories/{id}
DELETE /api/cost-centres/categories/{id}

```

List Cost Centres

```

GET /api/cost-centres

```

Create Cost Centre Form

Field	Input	Required	Key
Name	text	Yes	`name`
Category	select	Yes	`category_id` - from `GET /api/cost-centres/categories`
Description	textarea	No	`description`
Parent Cost Centre	select	No	`parent_id`

...

POST /api/cost-centres

Body: { "name": "Mumbai Branch", "category_id": 1 }

...

Edit/Delete

...

PUT /api/cost-centres/{id}

DELETE /api/cost-centres/{id}

...

3.7 Budgets

****Route:**** `/accounting/budgets`

List

...

GET /api/budgets?status=ACTIVE

...

Table: Name, Period Type, From-To, Status, Actions (View, Edit, Variance, Delete).

Create Budget Form

Field	Input	Required	Key
-------	-------	----------	-----

Budget Name	text	Yes	`name`
-------------	------	-----	--------

Period Type	select	Yes	`period_type` - ANNUAL / QUARTERLY / MONTHLY
-------------	--------	-----	--

From Date	date	Yes	`from_date`
-----------	------	-----	-------------

To Date	date	Yes	`to_date`
---------	------	-----	-----------

Notes	textarea	No	`notes`
-------	----------	----	---------

...

****Budget Lines (repeating rows):****

Field	Input	Required	Key
Ledger	searchable select	Yes	`ledger_id` - from `GET /api/ledgers`
Period Label	text	Yes	`period_label` e.g. "Apr-2025"
Budgeted Amount	number	Yes	`amount`

...

POST /api/budgets

Body: {

"name": "FY 2025-26 Annual Budget",

"period_type": "ANNUAL",

"from_date": "2025-04-01",

"to_date": "2026-03-31",

"items": [

{ "ledger_id": 5, "period_label": "FY2025-26", "amount": 500000 }

]

}

...

Variance Report (Budget vs Actual)

...

GET /api/budgets/{id}/variance

...

Table: Ledger, Budgeted Amount, Actual (from vouchers), Variance, Variance %.

Edit / Delete

...

PUT /api/budgets/{id}

DELETE /api/budgets/{id}

...

4. INVOICING & BILLING

4.1 Sales Invoices

****Route:**** `/invoicing/sales`

List

```
GET
/api/sales-invoices?page=1&limit=20&status=POSTED&from_date=2025-04-01&to_date=2026-03-31&customer_id=&payment_status=
```

Filters: Status, Payment Status, Date Range, Customer.

Table: Invoice No, Date, Customer, Net Total, Status, Payment Status, Actions.

Create Sales Invoice Form

Pre-load data (on form open):

```
GET /api/customers?limit=100          → customer dropdown
GET /api/settings/warehouses          → warehouse options
GET /api/items?limit=100              → items (lazy search recommended)
```

Header Section:

Field	Input	Required	Key	Notes
Customer	searchable select	Yes	<code>`customer_id`</code>	type-ahead search
Invoice Date	date	Yes	<code>`invoice_date`</code>	default: today
Due Date	date	No	<code>`due_date`</code>	default: invoice_date + 30 days
Invoice Type	select	No	<code>`invoice_type`</code>	B2B (default), B2C, EXPORT, SEZ
Place of Supply	select	Yes	<code>`place_of_supply`</code>	2-digit state code
Reference No	text	No	<code>`reference_number`</code>	

Place of Supply – India State Codes:

```
`01=JK, 02=HP, 03=PB, 04=CH, 05=UK, 06=HR, 07=DL, 08=RJ, 09=UP, 10=BR, 11=SK, 12=AR,
13=NL, 14=MN, 15=MZ, 16=TR, 17=ML, 18=AS, 19=WB, 20=JH, 21=OR, 22=CG, 23=MP, 24=GJ,
27=MH, 28=AP, 29=KA, 30=GA, 32=KL, 33=TN, 36=TS`
```

Item Lines Table (add rows dynamically):

Field	Input	Required	Key	Notes
Item	searchable select	Yes	<code>`item_id`</code>	auto-fills rate, gst_rate, hsn_code
Warehouse	select	Yes	<code>`warehouse_id`</code>	
Qty	number	Yes	<code>`qty`</code>	
Rate	number	Yes	<code>`rate`</code>	editable, pre-filled from item
Discount %	number	No	<code>`discount_pct`</code>	default 0

```
| HSN Code | text | No | `hsn_code` | auto-filled from item |
| GST Rate % | select | Yes | `gst_rate` | 0 / 5 / 12 / 18 / 28 |
| Taxable Amt | computed | - | `taxable_amount` | `qty × rate × (1 - discount/100)` |
| CGST / SGST or IGST | computed | - | - | see GST logic below |
| Total | computed | - | `total_amount` | `taxable + tax` |
```

****GST Auto-Calculation Logic:****

```
...
```

```
If place_of_supply === company.state_code:
```

```
    // Intra-state
```

```
    cgst_rate = gst_rate / 2
```

```
    sgst_rate = gst_rate / 2
```

```
    igst_rate = 0
```

```
Else:
```

```
    // Inter-state
```

```
    cgst_rate = 0
```

```
    sgst_rate = 0
```

```
    igst_rate = gst_rate
```

```
...
```

```
> Get `company.state_code` from the active company stored in localStorage/context.
```

****Footer (auto-computed):****

```
Gross Total → (-) Discount → Taxable Amount → (+) CGST → (+) SGST → (+) IGST → (+)
```

```
Cess → (±) Round Off → **Net Total**
```

****Save as DRAFT:****

```
...
```

```
POST /api/sales-invoices
```

```
Body: {
```

```
  "customer_id": 1,
```

```
  "invoice_date": "2026-03-15",
```

```
  "due_date": "2026-04-15",
```

```
  "invoice_type": "B2B",
```

```
  "place_of_supply": "27",
```

```
  "items": [
```

```
    {
```

```
      "item_id": 1,
```

```
      "warehouse_id": 1,
```

```
      "qty": 2,
```

```
      "rate": 55000,
```

```
      "discount_pct": 0,
```

```
      "gst_rate": 18,
```

```

        "hsn_code": "8471"
    }
]
}
...

**Action Buttons (status-based):**

| Button | Condition | API |
|---|---|---|
| Post Invoice | status=DRAFT | `POST /api/sales-invoices/{id}/post` |
| Collect Payment | status=POSTED, payment_status≠PAID | `POST
/api/sales-invoices/{id}/payment` |
| Submit for Approval | status=DRAFT | `POST /api/approvals` + entity |
| Cancel Invoice | status=DRAFT or POSTED | `POST /api/sales-invoices/{id}/cancel` |
| Print | any | frontend PDF generation |

**Collect Payment Form:**

| Field | Input | Required | Key |
|---|---|---|---|
| Amount | number | Yes | `amount` |
| Payment Date | date | Yes | `payment_date` |
| Payment Mode | select | Yes | `payment_mode` - CASH / BANK / CHEQUE / UPI / NEFT /
RTGS |
| Reference No | text | No | `reference` |

...

POST /api/sales-invoices/{id}/payment
Body: { "amount": 129800, "payment_date": "2026-03-20", "payment_mode": "BANK",
"reference": "NEFT/001" }
...

---

### 4.2 Purchase Invoices

**Route:** `/invoicing/purchases`

#### List
...

GET /api/purchase-invoices?page=1&limit=20&status=&vendor_id=&from_date=&to_date=
...

```

Create Purchase Invoice Form

Header:

Field	Input	Required	Key
Vendor	searchable select	Yes	`vendor_id`
Vendor Invoice No	text	No	`invoice_number`
Invoice Date	date	Yes	`invoice_date`
Due Date	date	No	`due_date`
Place of Supply	select	Yes	`place_of_supply`

Item Lines:

Same as Sales Invoice PLUS:

Field	Input	Key	Notes
ITC Eligibility	select	`itc_eligibility`	ELIGIBLE / INELIGIBLE / PARTIALLY_ELIGIBLE

Save:

```
POST /api/purchase-invoices
Body: { "vendor_id": 1, "invoice_date": "...", "place_of_supply": "27", "items": [...] }

```

Action Buttons:

Button	Condition	API
Post Invoice	DRAFT	`POST /api/purchase-invoices/{id}/post` - receives stock
Record Payment	POSTED	`POST /api/purchase-invoices/{id}/payment`
Cancel	DRAFT/POSTED	`POST /api/purchase-invoices/{id}/cancel`

4.3 Credit Notes (Sales Returns)

****Route:**** `/invoicing/credit-notes`

List

```
GET /api/credit-notes?page=1&limit=20
```

Create Credit Note Form

Field	Input	Required	Key
Customer	searchable select	Yes	`customer_id`
Credit Note Date	date	Yes	`credit_date`
Against Invoice (optional)	select	No	`original_invoice_id` - from `GET /api/sales-invoices?customer_id=X`
Place of Supply	select	Yes	`place_of_supply`

****Item Lines:**** Same as Sales Invoice.

POST /api/credit-notes

Body: {

```
"customer_id": 1,
"credit_date": "2026-03-20",
"original_invoice_id": 5,
"place_of_supply": "27",
"items": [{ "item_id": 1, "quantity": 2, "rate": 55000, "gst_rate": 18, "hsn_code": "8471" }]
```

}

****Action Buttons:****

Button	Condition	API
Post	DRAFT	`POST /api/credit-notes/{id}/post` - restores stock (IN)
Cancel	DRAFT/POSTED	`POST /api/credit-notes/{id}/cancel`

4.4 Debit Notes (Purchase Returns)

****Route:**** `/invoicing/debit-notes`

Create Debit Note Form

Field	Input	Required	Key

Vendor	searchable select	Yes	`vendor_id`
Debit Note Date	date	Yes	`debit_date`
Against Invoice	select	No	`original_invoice_id`
Place of Supply	select	Yes	`place_of_supply`

****Item Lines:**** Same as Purchase Invoice.

...

POST /api/debit-notes

Body: { "vendor_id": 1, "debit_date": "...", "original_invoice_id": 3, "place_of_supply": "27", "items": [...] }

...

****Action Buttons:****

Button	Condition	API
--- --- ---		
Post	DRAFT	`POST /api/debit-notes/{id}/post` - reduces stock (OUT)
Cancel	DRAFT/POSTED	`POST /api/debit-notes/{id}/cancel`

4.5 Proforma Invoices

****Route:**** `/invoicing/proforma`

Create Form

Same fields as Sales Invoice, plus:

Field	Input	Key
--- --- ---		
Valid Until	date	`valid_until`
Notes	textarea	`notes`

...

POST /api/proforma-invoices

Body: { "customer_id": 1, "proforma_date": "2026-03-01", "valid_until": "2026-03-31", "place_of_supply": "27", "items": [...] }

...

****Status Lifecycle Buttons:****

Current Status	Button	API
--- --- ---		
DRAFT	Send	`POST /api/proforma-invoices/{id}/send`


```
| SENT | Accept | `POST /api/proforma-invoices/{id}/accept` |
| SENT | Reject | `POST /api/proforma-invoices/{id}/reject` + `{ "reason": "..." }` |
| ACCEPTED | Convert to Invoice | `POST /api/proforma-invoices/{id}/convert` |
```

4.6 Sales Orders

****Route:**** `/invoicing/sales-orders`

Create Form

Field	Input	Required	Key
Customer	searchable select	Yes	`customer_id`
Order Date	date	Yes	`order_date`
Expected Delivery	date	No	`expected_delivery`

****Item Lines:****

Field	Input	Key
Item	searchable select	`item_id`
Qty	number	`qty`
Rate	number	`rate`

...

POST /api/sales-orders

Body: { "customer_id": 1, "order_date": "2026-03-15", "items": [{ "item_id": 1, "qty": 5, "rate": 55000 }] }

...

****Cancel:****

...

POST /api/sales-orders/{id}/cancel

...

4.7 Purchase Orders

****Route:**** `/invoicing/purchase-orders`

Create Form

Field	Input	Required	Key
Vendor	searchable select	Yes	`vendor_id`
PO Number	text	Yes	`po_number`
Order Date	date	Yes	`order_date`
Expected Delivery	date	No	`expected_delivery`
Notes	textarea	No	`notes`

Item Lines:

Field	Input	Key
Item	searchable select	`item_id`
Qty	number	`quantity`
Rate	number	`rate`
Amount	computed	`qty × rate`

...

POST /api/purchase-orders

Body: { "vendor_id": 1, "po_number": "PO-2026-001", "order_date": "2026-03-15",
"items": [...], "total_amount": 10000 }

...

Status Buttons:

Button	Condition	API
Confirm	DRAFT	`POST /api/purchase-orders/{id}/confirm`
Cancel	DRAFT/CONFIRMED	`POST /api/purchase-orders/{id}/cancel`

5. INVENTORY MANAGEMENT

5.1 Stock Items

****Route:**** `/inventory/items`

List

...

```
GET /api/items?page=1&limit=20&search=&category=
```

```
...
```

Table: SKU, Name, Unit, Sale Price, Purchase Price, Current Stock, Reorder Level.

Create Item Form

Field	Input	Required	Key	Notes
Item Name	text	Yes	`name`	
SKU / Code	text	No	`sku`	
Description	textarea	No	`description`	
Unit	select	Yes	`unit`	PCS / KG / LTR / MTR / BOX / SET
Category	text	No	`category`	
Sale Price	number	Yes	`sale_price`	
Purchase Price	number	No	`purchase_price`	
GST Rate %	select	Yes	`gst_rate`	0 / 5 / 12 / 18 / 28
HSN/SAC Code	text	No	`hsn_code`	lookup: `GET /api/hsn-sac?search=X`
Opening Stock	number	No	`opening_stock`	default 0
Reorder Level	number	No	`reorder_level`	alert threshold
Track Inventory	toggle	No	`track_inventory`	default true

```
...
```

```
POST /api/items
```

```
Body: { "name": "Laptop HP 15", "sku": "HP-001", "unit": "PCS", "sale_price": 55000,
"purchase_price": 45000, "gst_rate": 18, "hsn_code": "8471", "opening_stock": 10,
"reorder_level": 2 }
```

```
...
```

View Item Stock

```
...
```

```
GET /api/items/{id}/stock
```

```
...
```

Shows current stock per warehouse.

Edit / Delete

```
...
```

```
PUT /api/items/{id}
```

```
DELETE /api/items/{id}
```

```
...
```

```
---
```

5.2 Stock Summary

****Route:**** `/inventory/stock-summary``

...

GET `/api/inventory/stock-summary?warehouse_id=1`

...

or

...

GET `/api/reports/stock-summary?warehouse_id=1&category=Electronics`

...

Filters: Warehouse (select from `GET /api/settings/warehouses``), Category, Item.

Table: Item, Unit, Opening, IN Qty, OUT Qty, Closing, Value.

5.3 Stock Transfer

****Route:**** `/inventory/transfer``

Create Transfer Form

Field	Input	Required	Key
From Warehouse	select	Yes	<code>`from_warehouse_id`</code> - from <code>GET /api/settings/warehouses`</code>
To Warehouse	select	Yes	<code>`to_warehouse_id`</code>
Transfer Date	date	Yes	<code>`transfer_date`</code>
Item	searchable select	Yes	<code>`item_id`</code>
Quantity	number	Yes	<code>`qty`</code>
Remarks	text	No	<code>`remarks`</code>

...

POST `/api/inventory/transfer-stock`

Body: `{ "item_id": 1, "from_warehouse_id": 1, "to_warehouse_id": 2, "qty": 20, "transfer_date": "2026-03-20" }`

...

5.4 Wastage

****Route:**** `/inventory/wastage`

List

...

GET /api/wastage?page=1&limit=20&status=PENDING

...

Table: Date, Item, Warehouse, Qty, Reason, Status, Actions.

Create Form

Field	Input	Required	Key
Item	searchable select	Yes	`item_id`
Warehouse	select	Yes	`warehouse_id`
Quantity	number	Yes	`quantity`
Wastage Date	date	Yes	`wastage_date`
Reason	text	Yes	`reason`
Remarks	textarea	No	`remarks`

...

POST /api/wastage

Body: { "item_id": 1, "warehouse_id": 1, "quantity": 5, "wastage_date": "2026-03-01",
"reason": "Damaged" }

...

Manager Action Buttons:

Button	API	Body
Approve	`POST /api/wastage/{id}/approve`	`{ "remarks": "Verified" }` → creates stock OUT
Reject	`POST /api/wastage/{id}/reject`	`{ "remarks": "Not proven" }`

5.5 Stock Verification (Physical Count)

****Route:**** `/inventory/stock-verification`

List

...

GET /api/stock-verification

```

'''
Table: Date, Warehouse, Status (OPEN / SUBMITTED / POSTED), Actions.

#### Create Verification Form

| Field | Input | Required | Key |
|---|---|---|---|
| Warehouse | select | Yes | `warehouse_id` |
| Verification Date | date | Yes | `verification_date` |
| Notes | textarea | No | `notes` |

'''

POST /api/stock-verification
Body: { "warehouse_id": 1, "verification_date": "2026-03-31", "notes": "Year-end
count" }

'''

> Backend auto-fills `system_qty` for every item in the warehouse.

#### Verification Detail - Enter Physical Counts

'''

GET /api/stock-verification/{id}

'''

Show table with `system_qty` pre-filled. User enters `physical_qty` per row.

'''

POST /api/stock-verification/{id}/count
Body: { "counts": [{ "item_id": 1, "physical_qty": 48 }, { "item_id": 2,
"physical_qty": 95 }] }

'''

**View Variance Report:**

'''

GET /api/stock-verification/{id}/variance

'''

Table: Item, System Qty, Physical Qty, Variance (+ or -).

**Post Adjustments:**

'''

POST /api/stock-verification/{id}/post

'''

> Creates StockLedger IN/OUT entries to reconcile differences.

```

```

---

### 5.6 HSN / SAC Codes

**Route:** `/inventory/hsn-sac`

#### List
```
GET /api/hsn-sac?type=HSN&search=8471&page=1&limit=50
```

Filters: Type (HSN/SAC), Search Code or Description.
Table: Code, Type, Description, GST Rate %, Cess %.

#### Create Form



| Field       | Input    | Required | Key                 |
|-------------|----------|----------|---------------------|
| Code        | text     | Yes      | `code`              |
| Type        | select   | Yes      | `type` - HSN or SAC |
| Description | textarea | No       | `description`       |
| GST Rate %  | number   | No       | `gst_rate`          |
| Cess Rate % | number   | No       | `cess_rate`         |



```
POST /api/hsn-sac
Body: { "code": "8471", "type": "HSN", "description": "Computers", "gst_rate": 18,
"cess_rate": 0 }
```

#### Edit
```
PUT /api/hsn-sac/{id}
Body: { "gst_rate": 12 }
```

---

## 6. MANUFACTURING

---

### 6.1 Bill of Materials (BOM)

```

```
**Route:** `/manufacturing/bom`
```

List

```
...
```

```
GET /api/manufacturing/bom?status=ACTIVE
```

```
...
```

Table: Finished Item, Version, Qty per Batch, Status, Actions.

Create BOM Form

Field	Input	Required	Key
Finished Item	searchable select	Yes	`finished_item_id` – from `GET /api/items`
Version	text	Yes	`version` e.g. "1.0"
Qty Produced per batch	number	Yes	`qty_produced`
Status	select	No	`status` – DRAFT / ACTIVE / OBSOLETE

Raw Materials (repeating rows):

Field	Input	Required	Key	Notes
Raw Material Item	searchable select	Yes	`item_id`	
Qty Required	number	Yes	`qty`	per `qty_produced` units
Wastage %	number	No	`wastage_pct`	e.g. 2 for 2%
Sub-BOM	select	No	`sub_bom_id`	for multi-level BOM

```
...
```

```
POST /api/manufacturing/bom
```

```
Body: {
  "finished_item_id": 1,
  "version": "1.0",
  "qty_produced": 1,
  "items": [
    { "item_id": 2, "qty": 5, "wastage_pct": 2 },
    { "item_id": 3, "qty": 2, "wastage_pct": 0 }
  ]
}
```

```
...
```

BOM Explosion (Material Planning Tool)

Widget on BOM detail page: enter qty to produce → show material requirements.

...

GET /api/manufacturing/bom/{id}/explode?qty=100

...

Table: Raw Material, Required Qty (with wastage), Current Stock, Shortage.

6.2 Production Orders

****Route:**** `/manufacturing/production-orders`

List

...

GET /api/manufacturing/production-orders?status=DRAFT

...

Create Form

Field	Input	Required	Key
--- --- --- ---			
BOM	select	Yes	`bom_id` – from `GET /api/manufacturing/bom?status=ACTIVE`
Finished Item	auto-filled	Yes	`finished_item_id`
Qty to Produce	number	Yes	`qty_to_produce`
Output Warehouse	select	Yes	`warehouse_id`
Planned Start	date	No	`planned_start`
Planned End	date	No	`planned_end`
Notes	textarea	No	`notes`

...

POST /api/manufacturing/production-orders

Body: { "bom_id": 1, "finished_item_id": 1, "qty_to_produce": 100, "warehouse_id": 1, "planned_start": "2026-04-01", "planned_end": "2026-04-10" }

...

****Status Buttons:****

Button	Condition	API	Effect
--- --- --- ---			
Release	DRAFT	`POST /api/manufacturing/production-orders/{id}/release`	Consumes raw materials (stock OUT)

```
| Complete | RELEASED/IN_PROGRESS | `POST`  
/api/manufacturing/production-orders/{id}/complete` + `{ "qty_produced": 98 }` | Adds  
finished goods (stock IN) |
```

6.3 Job Work

****Route:**** `/manufacturing/job-work`

Create Form

Field	Input	Required	Key	Notes
Type	radio	Yes	`type`	OUT = send to vendor, IN = receive back
Vendor	searchable select	Yes	`vendor_id`	
Item	searchable select	Yes	`item_id`	
Qty	number	Yes	`qty_sent`	
Sent Date	date	Yes	`sent_date`	
Expected Return	date	No	`expected_return`	
Job Charges	number	No	`job_charges`	
Notes	textarea	No	`notes`	

...

POST /api/manufacturing/job-work

Body: { "type": "OUT", "vendor_id": 1, "item_id": 1, "qty_sent": 100, "sent_date":
"2026-03-01", "job_charges": 5000 }

...

7. GST & TAXATION

7.1 GST Summary Report

****Route:**** `/gst/summary`

Filters: From Date, To Date.

...

GET /api/reports/gst-summary?from_date=2025-04-01&to_date=2026-03-31

```
Display: GSTR-1 outward (sales) by HSN code, GSTR-2 inward (purchases), CGST/SGST/IGST column totals.
```

7.2 Reverse Charge Mechanism (RCM)

```
**Route:** `/gst/rcm`
```

```
Two tabs: **RCM Transactions** | **RCM Liability**
```

RCM Transactions (Unregistered Vendor Purchases)

```
GET /api/rcm/transactions?from_date=2025-04-01&to_date=2026-03-31&vendor_id=1
```

```
Table: Date, Vendor, Invoice No, Taxable Amount, IGST/CGST/SGST payable under RCM.
```

RCM Liability Summary

```
GET /api/rcm/liability?from_date=2025-04-01&to_date=2026-03-31
```

```
Summary card: Total IGST Payable, Total CGST Payable, Total SGST Payable, ITC Eligibility note.
```

8. PAYROLL & HR

8.1 Employees

```
**Route:** `/payroll/employees`
```

List

```
GET /api/payroll/employees?status=ACTIVE&department=Engineering&page=1&limit=20
```

```
Table: Code, Name, Department, Designation, DOJ, Salary, Status, Actions.
```

Create Employee Form

Field	Input	Required	Key
Employee Code	text	Yes	`employee_code`
Full Name	text	Yes	`name`
Email	email	No	`email`
Phone	text	No	`phone`
Department	text	No	`department`
Designation	text	No	`designation`
Date of Joining	date	Yes	`date_of_joining`
Basic Salary	number	Yes	`basic_salary`
PAN	text	No	`pan`
Aadhaar	text	No	`aadhaar`
PF Account No	text	No	`pf_account`
ESI No	text	No	`esi_number`
Bank Account No	text	No	`bank_account`
Bank IFSC	text	No	`bank_ifsc`
Bank Name	text	No	`bank_name`
Status	select	No	`status` - ACTIVE / INACTIVE / TERMINATED

...

POST /api/payroll/employees

Body: { "employee_code": "EMP001", "name": "Rahul Sharma", "date_of_joining": "2025-01-01", "basic_salary": 50000, "department": "Engineering", ... }

...

8.2 Attendance

****Route:**** `/payroll/attendance`

Monthly Calendar View

...

GET /api/payroll/attendance/{employee_id}?month=2026-03

...

Show each day's status per employee: PRESENT (green), ABSENT (red), HALF_DAY (orange), LEAVE (blue), HOLIDAY (gray), WEEK_OFF (gray).

Mark Attendance Form

Field	Input	Required	Key	Values
-------	-------	----------	-----	--------

```

|---|---|---|---|
| Employee | select | Yes | `employee_id` | |
| Date | date | Yes | `attendance_date` | |
| Status | select | Yes | `status` | PRESENT / ABSENT / HALF_DAY / LEAVE / HOLIDAY /
WEEK_OFF |
| Check In | time | No | `check_in` | |
| Check Out | time | No | `check_out` | |

...

POST /api/payroll/attendance
Body: { "employee_id": 1, "attendance_date": "2026-03-15", "status": "PRESENT",
"check_in": "09:00", "check_out": "18:00" }
...

#### Bulk Attendance (for a whole day)
...

POST /api/payroll/attendance/bulk
Body: {
  "attendance_date": "2026-03-15",
  "records": [
    { "employee_id": 1, "status": "PRESENT" },
    { "employee_id": 2, "status": "ABSENT" }
  ]
}
...

---

### 8.3 Salary Structure

**Route:** `/payroll/salary-structure`

#### View Employee's Structure
...

GET /api/payroll/employees/{id}/salary-structure
...

#### Set / Edit Salary Structure Form

| Field | Input | Required | Key |
|---|---|---|---|
| Employee | select (pre-selected from list) | Yes | `employee_id` |

```

```

| Basic | number | Yes | `basic` |
| HRA | number | No | `hra` |
| Special Allowance | number | No | `special_allowance` |
| Other Allowance | number | No | `other_allowance` |
| Medical Allowance | number | No | `medical_allowance` |
| Transport Allowance | number | No | `transport_allowance` |

**Auto-Calculated (show as read-only below):**
| Deduction | Formula |
|---|---|
| PF (Employee) | 12% of basic, capped at ₹1,800 (if gross > ₹15,000) |
| PF (Employer) | Same as employee PF |
| ESI (Employee) | 0.75% of gross (only if gross ≤ ₹21,000) |
| ESI (Employer) | 3.25% of gross |
| Professional Tax | ₹200 if gross > ₹15k / ₹150 if > ₹10k / ₹75 if > ₹7.5k |
| Net Salary | Total Earnings - Total Deductions |

...

POST /api/payroll/salary-structure
Body: { "employee_id": 1, "basic": 50000, "hra": 20000, "special_allowance": 10000,
"other_allowance": 5000 }
...

---

### 8.4 Payslip Generation

**Route:** `/payroll/payslips`

#### List
...

GET /api/payroll/payslips?month=2026-03&status=DRAFT&employee_id=
...

Table: Employee, Month, Gross, Deductions, Net Pay, Status, Actions.

#### Generate Payslip Form

| Field | Input | Required | Key |
|---|---|---|---|
| Employee | select | Yes | `employee_id` |
| Salary Month | month-picker | Yes | `salary_month` - format YYYY-MM |

```

```
POST /api/payroll/payslips/generate
```

```
Body: { "employee_id": 1, "salary_month": "2026-03" }
```

```
> Backend pro-rates: `net_pay = net_salary * (present_days / working_days)`
```

****Status Buttons:****

```
| Button | Condition | API |
```

```
|---|---|---|
```

```
| Approve | DRAFT | `POST /api/payroll/payslips/{id}/approve` |
```

```
| Mark Paid | APPROVED | `POST /api/payroll/payslips/{id}/mark-paid` |
```

Payslip Print View

Show: Employee details, month, earnings table, deductions table, net pay, employer contributions, signature line.

```
---
```

9. BANKING & PAYMENTS

```
---
```

9.1 Post-Dated Cheques

```
**Route:** `/banking/cheques`
```

List

```
---
```

```
GET /api/cheques?status=PENDING&type=RECEIVED&page=1
```

```
---
```

```
**Due Soon Alert Banner:**
```

```
---
```

```
GET /api/cheques/due-soon?days=7
```

```
---
```

Show yellow warning banner with count of cheques due in next 7 days.

Filters: Type (RECEIVED/ISSUED), Status, Party, Date range.

Table: Cheque No, Bank, Party, Amount, Cheque Date, Status, Actions.

Create Cheque Form

```

| Field | Input | Required | Key | Notes |
|---|---|---|---|---|
| Type | radio | Yes | `type` | RECEIVED (from customer) / ISSUED (to vendor) |
| Party Type | select | Yes | `party_type` | CUSTOMER / VENDOR |
| Party | searchable select | Yes | `party_id` | load customers or vendors based on
party_type |
| Cheque Number | text | Yes | `cheque_number` | |
| Bank Name | text | No | `bank_name` | |
| Cheque Date | date | Yes | `cheque_date` | future date = post-dated |
| Amount | number | Yes | `amount` | |
| Remarks | text | No | `remarks` | |

...

POST /api/cheques
Body: { "type": "RECEIVED", "party_type": "CUSTOMER", "party_id": 1, "cheque_number":
"CHQ001", "cheque_date": "2026-04-15", "amount": 50000 }
...

**Action Buttons:**

| Button | Condition | API | Extra Body |
|---|---|---|---|
| Clear | status=PENDING | `POST /api/cheques/{id}/clear` | `{ "cleared_date":
"2026-04-15", "remarks": "Deposited" }` |
| Bounce | status=PENDING | `POST /api/cheques/{id}/bounce` | `{ "reason":
"Insufficient funds", "charge": 500 }` |
| Cancel | status=PENDING | `POST /api/cheques/{id}/cancel` | - |

---

## 10. REPORTS & MIS

**Route prefix:** `/reports/`

> All reports share a **Filter Bar** component at the top and a **Download/Print**
button.

---

### 10.1 Trial Balance

**Route:** `/reports/trial-balance`
...

```



```
GET /api/reports/trial-balance?from_date=2025-04-01&to_date=2026-03-31
```

```
...
```

Filters: From Date, To Date.

Display: Ledger Group/Ledger tree, Opening DR, Opening CR, Transactions DR, Transactions CR, Closing DR, Closing CR.

```
---
```

10.2 Profit & Loss Account

****Route:**** `/reports/profit-loss``

```
...
```

```
GET /api/reports/profit-loss?from_date=2025-04-01&to_date=2026-03-31
```

```
...
```

Filters: From Date, To Date.

Display: Income section | Expenses section | Net Profit/Loss at bottom.

```
---
```

10.3 Balance Sheet

****Route:**** `/reports/balance-sheet``

```
...
```

```
GET /api/reports/balance-sheet?as_on_date=2026-03-31
```

```
...
```

Filters: As On Date.

Display: Assets (left/top) | Liabilities+Capital (right/bottom). Must balance.

```
---
```

10.4 Ledger Account Report

****Route:**** `/reports/ledger``

```
...
```

```
GET /api/reports/ledger/{ledger_id}?from_date=2025-04-01&to_date=2026-03-31
```

```
...
```

Filters: Ledger (searchable select from ``GET /api/ledgers``), From Date, To Date.

Display: Date, Voucher No, Narration, DR Amount, CR Amount, Running Balance.

```
---
```

10.5 Outstanding Receivables

****Route:**** `/reports/outstanding-receivables``

```
...
```

```
GET /api/reports/outstanding-receivables?as_on_date=2026-03-31&customer_id=1
```

```
Filters: As On Date, Customer (optional).
```

```
Table: Customer, Invoice No, Invoice Date, Due Date, Invoice Amt, Paid, Outstanding,  
Overdue Days.
```

```
---
```

10.6 Outstanding Payables

```
**Route:** `/reports/outstanding-payables`
```

```
---
```

```
GET /api/reports/outstanding-payables?as_on_date=2026-03-31&vendor_id=1
```

```
---
```

```
Filters: As On Date, Vendor (optional).
```

```
---
```

10.7 Stock Summary Report

```
**Route:** `/reports/stock-summary`
```

```
---
```

```
GET /api/reports/stock-summary?warehouse_id=1
```

```
---
```

```
Filters: Warehouse (optional).
```

```
Table: Item, Unit, Opening Qty, IN, OUT, Closing Qty, Value.
```

```
---
```

10.8 Sales Analysis

```
**Route:** `/reports/sales-analysis`
```

```
---
```

```
GET /api/reports/sales-analysis?from_date=2025-04-01&to_date=2026-03-31&group_by=month
```

```
---
```

```
`group_by`: `month` or `customer`
```

```
Display: Bar chart + summary table.
```

```
---
```

10.9 Purchase Analysis

```
**Route:** `/reports/purchase-analysis`
```

```
---
```

```
GET
```

```
/api/reports/purchase-analysis?from_date=2025-04-01&to_date=2026-03-31&group_by=month
```

```
---
```

```
`group_by`: `month` or `vendor`
```

```
---
```

10.10 GST Summary

```
**Route:** `/reports/gst-summary`
```

```
...
```

```
GET /api/reports/gst-summary?from_date=2025-04-01&to_date=2026-03-31
```

```
...
```

Shows: GSTR-1 summary (sales by GST rate), GSTR-2 summary (purchases).

```
---
```

10.11 Cash Flow Report

```
**Route:** `/reports/cash-flow`
```

```
...
```

```
GET /api/reports/cash-flow?from_date=2025-04-01&to_date=2026-03-31
```

```
...
```

```
---
```

10.12 Fund Flow Report

```
**Route:** `/reports/fund-flow`
```

```
...
```

```
GET /api/reports/fund-flow?from_date=2025-04-01&to_date=2026-03-31
```

```
...
```

```
---
```

10.13 Cash / Bank Book

```
**Route:** `/reports/cash-bank-book`
```

```
...
```

```
GET /api/reports/cash-bank-book?from_date=2026-03-27&to_date=2026-03-27&ledger_id=1
```

```
...
```

Filter: Ledger (only show Cash-in-Hand + Bank Accounts group ledgers).

```
---
```

10.14 Day Book

```
**Route:** `/reports/day-book`
```

```
...
```

```
GET /api/reports/day-book?from_date=2026-03-27&to_date=2026-03-27
```

```
Shows all vouchers for the date.
```

```
---
```

11. MULTI-CURRENCY

****Route:**** `/currencies``

List

```
...
```

```
GET /api/currencies
```

```
...
```

Table: Code, Name, Symbol, Exchange Rate (vs INR), Last Updated, Actions.

Add Currency Form

Field	Input	Required	Key
Currency Code	text	Yes	<code>`code`</code> – ISO 4217 e.g. USD
Name	text	Yes	<code>`name`</code>
Symbol	text	Yes	<code>`symbol`</code>
Exchange Rate (vs INR)	number	Yes	<code>`exchange_rate`</code>

```
...
```

```
POST /api/currencies
```

```
Body: { "code": "USD", "name": "US Dollar", "symbol": "$", "exchange_rate": 83.50 }
```

```
...
```

Update Rate

```
...
```

```
PATCH /api/currencies/{id}/rate
```

```
Body: { "exchange_rate": 84.25 }
```

```
...
```

Currency Converter Widget (inline on list page)

```
...
```

```
GET /api/currencies/convert?from=USD&to=INR&amount=1000
```

```
...
```

```
---
```

12. APPROVAL WORKFLOW

****Route:**** `/approvals`

Two Tabs: All Requests | My Pending

...

GET /api/approvals?status=PENDING&entity_type=SALES_INVOICE → All Requests

GET /api/approvals/pending → Requests I need to

act on

...

Table: Entity Type, Entity Ref, Requested By, Date Submitted, Remarks, Status, Actions.

Submit for Approval

> Place "Submit for Approval" button on Invoice/Voucher detail pages when status = DRAFT.

****Entity Types:**** `VOUCHER` / `SALES\_INVOICE` / `PURCHASE\_INVOICE` / `EXPENSE`

...

POST /api/approvals

Body: { "entity_type": "SALES_INVOICE", "entity_id": 5, "remarks": "Please approve" }

...

Approve (show only if current user ≠ requestor)

...

POST /api/approvals/{id}/approve

Body: { "remarks": "Looks good, approved" }

...

Reject

...

POST /api/approvals/{id}/reject

Body: { "reason": "Amount exceeds budget" }

...

> ****Important:**** Backend returns 403 if you try to approve your own request. Show friendly message: "You cannot approve a request you submitted."

13. SETTINGS

****Route prefix:**** `/settings/`

13.1 Company Settings

...

GET /api/settings/company → display

PUT /api/settings/company Body: { gstin, state_code, ... }

...

13.2 Voucher Types

...

GET /api/settings/voucher-types

...

Read-only list. 9 types: Payment, Receipt, Journal, Contra, Sales, Purchase, Credit Note, Debit Note, Stock Journal.

13.3 Warehouses

...

GET /api/settings/warehouses

POST /api/settings/warehouses Body: { "name": "Delhi Warehouse", "location": "Okhla, Delhi" }

PUT /api/settings/warehouses/{id} Body: { "name": "Updated Name", "location": "New Location" }

...

14. MASTER DATA (Customers & Vendors)

Customers

```
**Route:** `/master/customers`
```

Create Customer Form

Field	Input	Required	Key
--- --- --- ---			
Name	text	Yes	`name`
GSTIN	text	No	`gstIN`
Phone	text	No	`phone`
Email	email	No	`email`
Address	textarea	No	`address`
City	text	No	`city`
State	text	No	`state`
State Code	text	No	`state_code`
Opening Balance	number	No	`opening_balance`
Balance Type	radio	No	`balance_type` - DR (default) or CR
Credit Limit	number	No	`credit_limit`
Payment Terms (days)	number	No	`payment_terms`

```
...
```

```
POST /api/customers
```

```
GET /api/customers?page=1&limit=20&search=rajesh
```

```
PUT /api/customers/{id}
```

```
DELETE /api/customers/{id}
```

```
...
```

```
---
```

Vendors

```
**Route:** `/master/vendors`
```

Same form as Customer, `balance\_type` defaults to CR.

```
...
```

```
POST /api/vendors
```

```
GET /api/vendors?page=1&limit=20&search=sharma
```

```
PUT /api/vendors/{id}
```

```
DELETE /api/vendors/{id}
```

```
...
```

```
---
```

15. AUTHENTICATION

Login Page

...

POST /api/auth/login

Body: { "email": "...", "password": "..."}
...

Store in localStorage: `accessToken`, `refreshToken`, `user.id`, `user.company\_id`,
`user.name`.

Register Flow (New Business)

1. `POST /api/onboarding/register-owner` - creates owner account
2. `POST /api/auth/login` - get token
3. `POST /api/onboarding/company` - create company + seed all master data

Get Profile (on app boot to restore session)

...

GET /api/auth/profile

...

Returns: `{ user, company, companies[], roles[] }`

Logout

...

POST /api/auth/logout

Body: { "refresh_token": "..."}
...

Clear localStorage, redirect to login.

Switch Company

...

POST /api/auth/switch-company

Body: { "company_id": 2 }
...

Replace `accessToken` in storage with new one from response.

16. Common UI Components

Searchable Select (Autocomplete)

For Items, Customers, Vendors, Ledgers - implement type-ahead:

1. User types → debounce 300ms
2. Call `GET /api/{endpoint}?search=<query>&limit=20`
3. Render options dropdown

Status Badge Colors

```css

DRAFT / PENDING / OPEN → yellow (#F59E0B)  
POSTED / ACTIVE / CONFIRMED / COMPLETED / APPROVED → green (#10B981)  
CANCELLED / REJECTED / OBSOLETE / TERMINATED → red (#EF4444)  
PAID / CLEARED → blue (#3B82F6)  
PARTIALLY\_PAID / IN\_PROGRESS / HALF\_DAY → orange (#F97316)  
PRESENT → green  
ABSENT → red  
HALF\_DAY → orange  
LEAVE / HOLIDAY / WEEK\_OFF → gray (#6B7280)

```

Error Handling Toast Messages

`error_code`	Display
--- ---	
`VALIDATION_ERROR`	Show field-level errors from `errors[]` array
`DUPLICATE_ENTRY`	"{field} already exists"
`AUTHENTICATION_ERROR`	Redirect to /login
`AUTHORIZATION_ERROR`	"You don't have permission to do this"
`NOT_FOUND`	"Record not found"
`INTERNAL_ERROR`	"Something went wrong, please try again"
`FOREIGN_KEY_ERROR`	"This record is linked to other data and cannot be deleted"

Pagination Component

Always use `page` + `limit` query params.

Show: "Showing {(page-1)*limit + 1}-{min(page*limit, total)} of {total} records"

Previous / Next buttons. Disable Previous when page=1, Next when page=total_pages.

Date Range Filter (reusable component)

Default values: From = start of current financial year, To = today.

Store in component state; pass as query params on fetch.

API Quick Reference

Module	Base API Path
--------	---------------

```
|---|---|
| Auth | `/api/auth` |
| Onboarding | `/api/onboarding` |
| Company | `/api/company` |
| Users | `/api/users` |
| Roles | `/api/roles` |
| Customers | `/api/customers` |
| Vendors | `/api/vendors` |
| Items | `/api/items` |
| Ledger Groups | `/api/ledger-groups` |
| Ledgers | `/api/ledgers` |
| Vouchers | `/api/vouchers` |
| Trial Balance | `/api/trial-balance` |
| Sales Invoices | `/api/sales-invoices` |
| Purchase Invoices | `/api/purchase-invoices` |
| Credit Notes | `/api/credit-notes` |
| Debit Notes | `/api/debit-notes` |
| Proforma Invoices | `/api/proforma-invoices` |
| Sales Orders | `/api/sales-orders` |
| Purchase Orders | `/api/purchase-orders` |
| Inventory | `/api/inventory` |
| Wastage | `/api/wastage` |
| Stock Verification | `/api/stock-verification` |
| HSN/SAC | `/api/hsn-sac` |
| Manufacturing | `/api/manufacturing` |
| Payroll | `/api/payroll` |
| Bank Reconciliation | `/api/bank-reconciliations` |
| Cheques | `/api/cheques` |
| Interest | `/api/interest` |
| Cost Centres | `/api/cost-centres` |
| Budgets | `/api/budgets` |
| Currencies | `/api/currencies` |
| Dashboard | `/api/dashboard` |
| RCM | `/api/rcm` |
| Approvals | `/api/approvals` |
| Backups | `/api/backups` |
| Reports | `/api/reports` |
| Settings | `/api/settings` |
```

Document version: 2026-03-27 | Atyani ERP Backend v2.0

